

It's 2012, So Why Am I Driving a 1972 Bug?

Bryan Reinicke

University of North Carolina Wilmington



Several approaches that could be implemented on any platform via any methodology would greatly increase the odds of the successful deployment of a working information system.

One of the greatest success stories in automotive history has to be the Volkswagen Beetle.

From 1945 to 2003, VW turned out 21,529,464 Beetles worldwide. The key to the brand's incredible longevity was that, at its core, the Beetle did what it was supposed to do: provide reasonably cheap and reliable transportation.

Over the years, VW made several changes, from removing the crank start in 1949 to adding the front vent windows in 1952 to changing from a 24-HP, 1.1-liter engine to a 57-HP, 1.6-liter model in 1970. The company performed these tweaks to address flaws such as poor handling, the need for better airflow in the cabin, and a lack of power, but it never veered far from the car's distinctive shape and style or its 1930s-era chassis.

What does this have to do with systems analysis and design? With the advent of the traditional waterfall systems development life cycle (SDLC), developers gained a clear template for how to design and implement a new system. Over the years, several new methodolo-

gies have emerged, many of which seem to distance themselves from the SDLC and its problems, despite the fact that they all aim to do the same thing—develop and deploy a working information system. But these various methodologies tend to make cosmetic changes, rather than address any SDLC shortcomings. In essence, we're repackaging the car without addressing its known mechanical issues.

COMPARING METHODOLOGIES

The software development methodologies currently in widespread use include the following: waterfall, object-oriented analysis and design (OOAD), prototyping, rapid application development, and spiral. Their phases tend to overlap in some areas because, at their most basic level, each method must identify the problem to be solved, determine the best solution, and then build, test, and implement that solution.

To demonstrate the extent of this overlap, Table 1 compares phase definitions within the SDLC and OOAD

models, specifically, the waterfall model and the Unified Process (UP).

Other researchers have compared multiple development methodologies and found similarities that helped with categorization (S. Sircar et al., "Revolution or Evolution? A Comparison of Object-Oriented and Structured Systems Development Methods," *MIS Quarterly*, vol. 25, no. 4, 2001, pp. 457-471). Additional studies concluded that the differences were structural rather than representative of a radical change in approach (J. Iivari et al., "A Dynamic Framework for Classifying Information Systems Development Methodologies and Approaches," *J. Management Information Systems*, vol. 17, no. 3, 2001, pp. 179-218).

OVERCOMING FAILURE

Of the methodologies presented in Table 1, the original SDLC was more a predictive, rather than adaptive, methodology—that is, it assumed that all requirements were known at the beginning of the project and that they didn't change over time. In fact, the textbook definition of the waterfall methodology notes that the

Table 1. A comparison of the stages and definitions of the waterfall model and the Unified Process.

Traditional software development life cycle (waterfall model)	Definition	Object-oriented analysis and design (Unified Process model)	Definition
Planning	Understand the business problem, determine how to address it, and develop a project plan.	Inception	Develop and refine the vision for the system; make the business case and define the scope.
Analysis	Analyze the needs of the users and stakeholders; develop functional requirements for the new system and logical models of existing systems.	Elaboration	Move through multiple iterations of the system core's analysis, design, and implementation.
Logical design	Develop the new system's complete specifications and logical models.	Construction	Continue the system's design and implementation, with the focus moving from the core to other functions.
Physical design	Determine the physical design for the hardware and software to be implemented.		
Implementation	Acquire all hardware and software, test the new system, and move the system into production.	Transition	Test the system and move it into production.
Maintenance	Monitor, evaluate, and maintain the new system.		

requirements are frozen at the end of each phase and not revisited. Hence the term “waterfall,” which, as anyone who has been whitewater rafting can testify, makes it impossible to paddle back upstream.

The UP addresses this known flaw by emphasizing the flow of activities across the various phases. UP models emphasize the importance of continuing activities in design, management, and configuration across the stages, while the waterfall methodology does not. In addition, the UP also introduces an entirely new way of representing systems, specifically, the use of Unified Modeling Language.

Although the UP's use of Unified Modeling Language makes it easier to manipulate, it isn't clear that introducing a new modeling technique improves the odds for a successful systems implementation, especially because most problems are managerial rather than representational. To prove this point, consider the following list of factors essential for success (S.A. Brown et al., “What Matters When Introducing New Information Technology,” *Comm. ACM*, vol. 50, no. 9, 2007, pp. 91-96):

- *commitment*, or dedicating the financial and human resources necessary to complete a project;
- *knowledge*, or having IT personnel in place who know the business, and vice versa;
- *communication*, or the exchange of information among users, managers, and technology professionals;
- *planning*, or the strategic designs behind building and implementing systems; and
- *adequate infrastructure*, or the technical platform on which the system will run, including hardware, software, and telecommunications.

None of these factors is new or truly technical. Granted, the infrastructure is a technical detail, but with proper planning, the people involved in the implementation should know if the technical underpinnings are insufficient. Is it Windows 7's fault that it's difficult to install on a PC that's more than five years old?

In fact, the UP, prototyping, and agile methods were all developed, in part, to address some of these underlying issues by approaching

development in a different way. For example, UP presents the tasks to be completed during a software project as stretching over the project's entire length, whereas the more traditional SDLC approaches tasks as only occupying a portion of the project.

Agile methods and prototyping try to address some of these issues—namely, commitment, communication, and planning—by speeding up the development process and engaging users more directly (C. Larman and V.R. Basili, “Iterative and Incremental Development: A Brief History,” *Computer*, June 2003, pp. 47-56).

Agile methods have been in use for some time now and are very popular—and applicable—to many software projects related to Web-based development and design (F. Maurer and S. Martel, “Extreme Programming: Rapid Development for Web-Based Applications,” *IEEE Internet Computing*, vol. 6, no. 1, 2002, pp. 86-90). Agile methodologies increased the speed of development, and the iterative testing prescribed for these methods can be beneficial, but they can't be applied to every system, and they don't address all of the common problems identified with

failure. Despite improvements, failed systems projects and implementations are still distressingly common.

In fact, we could argue that, to a certain extent, agile methods can actually contribute to some problems. For instance, one of the rules of agile development is “focus on content, rather than representation.” This is intended to get developers to focus on developing the system, rather than developing better diagrams of the system, but it bleeds over into other areas, leading developers to focus on data content, rather than presentation to the user.

Certainly, content is critical, but poor data presentation can doom a project. I recently asked a developer who was purportedly using a prototyping methodology in an agile development shop if he had presented his prototype to users for feedback. His response was that “users don’t know what they want. I’ll show it to them when I’m done.” This attitude doesn’t endear developers to the user community, nor does it address the need for communication—a prime reason for failure.

THE DEVIL’S IN THE DETAILS

The difference between system success and failure is often in the details. The original Apple iPad and Motorola Xoom tablets, for example, were very similar at a hardware level, but one outsold the other by a wide margin. Many Xoom tablet reviews commented that the software felt “unfinished” compared to the iPad. I’m certain the developers felt it was finished—but users didn’t agree.

If nothing else, this example highlights the fact that developing for new mobile technologies, such as tablets, is no different from developing for old technologies, such as mainframes. The same criteria exist for success or failure, regardless of platform, so the more we treat this as undiscovered territory, the more likely we are to fail.

Each of the methodologies discussed here has a set of guidelines

to go along with it, many of which are presented as mutually exclusive. Thus, they introduce multiple ways of addressing known issues, but they fail to offer solutions that could be applied to any project regardless of methodology. This might help explain why research has found that methodologies are likely to be customized to fit in a given environment: developing a new methodology can’t address the environment for every possible information systems project (B. Fitzgerald, “The Use of Systems Development Methodologies in Practice: A Field Study,” *Information Systems J.*, vol. 7, 1997, pp. 201-212).

Developing for new mobile technologies, such as tablets, is no different from developing for old technologies, such as mainframes.

The ability to address actual problems, rather than just putting new packaging on the same product, is even more critical as IT moves back to the mainframe. Regardless of what we call it, the move to the cloud is simply a return to centralized storage and processing. Before we get carried away with thinking that the cloud is a never-before-explored technology in which storage and processing are centralized and thin clients only present data, we should revisit some of the problems that previously existed under this centralized schema. Dumb terminals might display more colors than just green or orange these days, but the concept is exactly the same.

Do the executives pushing for cloud-based solutions actually understand the risks of pushing sensitive corporate data in the cloud? What about the infrastructure requirements for maintaining up time? Even Amazon and Microsoft seem to have underestimated this, and they’re

considered to be technology-savvy companies. How does the cloud fit into the planning that should drive strategic information systems? Just because something’s “new” doesn’t mean that the requirements for success are.

So what can be done? If we know that a car has problems, simply changing the body work over and over again won’t fix them—we must address the underlying issues. In other words, we don’t need to learn another methodology, we need to fix the problems. Here are some approaches to help guide us:

- *Learn from our failures.* There’s an understandable tendency to try to sweep a failed system under the rug and forget about it. But if a system fails, we can learn from it. Those who fail to learn from history are doomed to repeat it.
- *Apply lessons across platforms.* The same things that caused a system written in Cobol to fail will cause apps written in Objective C to fail.
- *Make sure that developers know what the business is doing, and vice versa.* Too often, these groups are separated by multiple barriers. One option is to rotate IS and other managers through business and technological functions. This would force the managers to better appreciate what goes on in other areas.
- *Involve knowledgeable people from every area in the planning process.* According to the old adage, “selling isn’t implementing.” Make sure that the people who are in a position to evaluate and buy systems know what they’re doing.
- *The most important person in any systems implementation isn’t the developer, database designer, or*

even the user—it's the person who controls the budget. A more senior executive might not completely understand the project, but it's critically important for this person to be involved, educated, and excited about the project. If it's simply another expense line item to that person, the project is much more likely to be cut when the budget needs to be tightened.

Each of these approaches could be used on any project, implemented on any platform via any methodology, and would greatly increase the odds of the information system being successfully implemented. After all, regardless of the bodywork, it's still a car. **C**

Bryan Reinicke is an assistant professor of information systems at the University of North Carolina Wilmington. Contact him at reinickeb@uncw.edu.

Editor: Mike Hinchey, Lero—The Irish Software Engineering Research Centre;
mike.hinchey@lero.ie



Selected CS articles and columns are available for free at <http://ComputingNow.computer.org>.



Now available: **A video introducing the IEEE Computer Society's new OnlinePlus™ publication model for Transactions.**

Viewers will see an overview of the great features and benefits included with an OnlinePlus™ subscription and will take a tour of the user-friendly interface included on the accompanying disc.

Go to www.computer.org/onlineplus to view the video and learn all about it today.



IEEE  computer society

ONLINE  PLUS™
publishing evolved

computing

in SCIENCE & ENGINEERING

Subscribe today for the latest in computational science and engineering research, news and analysis, CSE in education, and emerging technologies in the hard sciences.

AIP

www.computer.org/cise

IEEE  computer society